Motor Health Monitoring Controller

STM32 Planning & Project Documentation

Embedded Engineer Intern Technical Task — Task 3

Submitted by: Surya K
Electronics and Communication Engineering
Sri Venkateswara College of Engineering, Chennai

Table of Contents

1. Introduction

This document presents the firmware planning and system documentation for the Motor Health Monitoring Controller, developed as part of the Embedded Engineer Intern technical assessment at OliveIoT Innovations Private Limited. The system is designed around a two-tier architecture: an STM32 microcontroller handles local sensing, control, and safety logic, while an ESP32 module bridges the system to a Firebase Realtime Database for remote monitoring and control.

The objective of the controller is to continuously monitor the operating health of an industrial motor by tracking current draw, temperature, and vibration, and to take protective action (alarm, buzzer, fault indication, and eventually motor stop) when unsafe operating conditions are detected. The system also tracks cumulative runtime and exposes both telemetry and remote command capability through the cloud layer.

1.1 Scope of This Document

- STM32 firmware planning: block diagram, GPIO allocation, ADC input plan, runtime logic, motor health logic, and fault logic
- ESP32 Firebase integration: data upload schema, command read schema, and simulated outputs
- Reference implementation code as developed and compiled for this submission
- Design notes and recommendations for production hardening

2. System Block Diagram

The block diagram below shows the complete signal flow: five sensing/input sources feed the STM32, which executes the health and fault logic and drives five control outputs. Telemetry and commands are exchanged with an ESP32 over UART, which in turn synchronizes with Firebase over Wi-Fi.

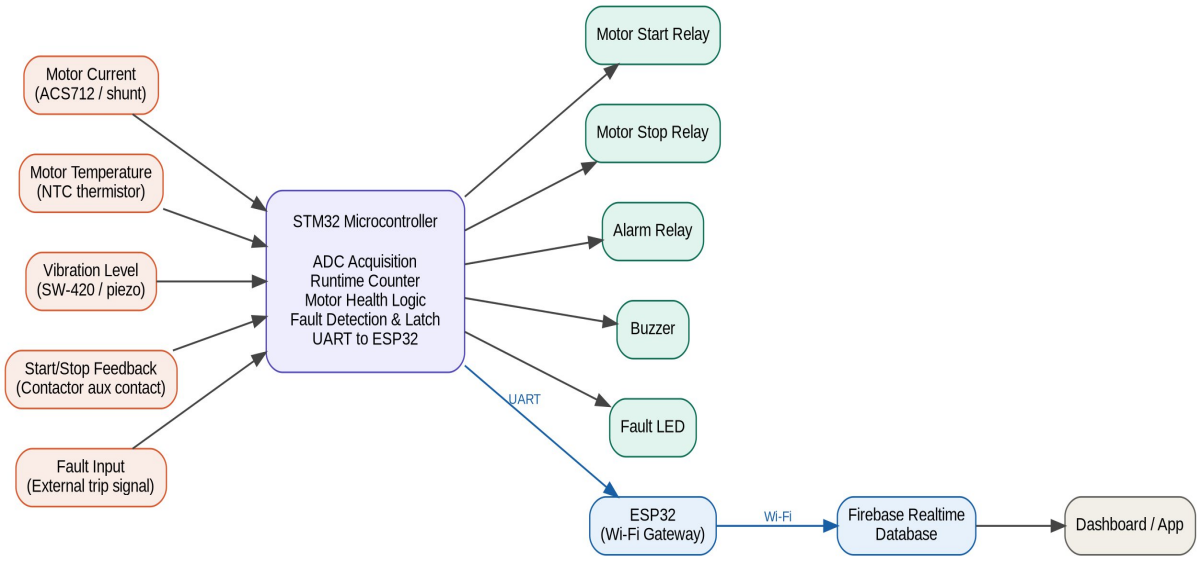

Figure 1: System block diagram — sensing, STM32 processing, control outputs, and cloud bridge

2.1 Architecture Notes

- Sensing side uses three analog channels (current, temperature, vibration) and two digital inputs (start/stop feedback, fault input).
- All digital field inputs use fail-safe (normally-closed) wiring conventions so that a broken wire is indistinguishable from an active fault.
- STM32 is the safety-critical decision-making core; ESP32 is purely a communication gateway and does not participate in protective logic.
- UART was selected over I2C/SPI for the STM32-ESP32 link due to its simplicity and noise immunity over board-to-board wiring.

3. GPIO Allocation Table

Pin assignments below follow a deliberate grouping convention: Port A is reserved for analog inputs and UART, Port B for digital field inputs, and Port C for digital outputs. This keeps the planning document easy to audit and maps cleanly to common STM32F1/F4 development boards.

#	Signal	Pin	Direction	Mode	Active Level	Notes
1	Motor Current	PA0	Input (analog)	ADC1_IN0	Analog	From ACS712 / shunt + signal conditioning
2	Motor Temperature	PA1	Input (analog)	ADC1_IN1	Analog	NTC thermistor voltage divider
3	Vibration Level	PA2	Input (analog)	ADC1_IN2	Analog	SW-420 analog output or piezo + conditioning
4	Start/Stop Feedback	PB0	Input (digital)	GPIO_PULLUP	Active-low	Opto-isolated from contactor aux contact, 50 ms debounce
5	Fault Input	PB1	Input (digital)	GPIO_PULLUP	Active-high	NC contact to GND, fail-safe on wire break
6	Motor Start Relay	PC0	Output (digital)	GPIO_OUTPUT_PP	Active-high	Drives start relay via transistor / driver IC
7	Motor Stop Relay	PC1	Output (digital)	GPIO_OUTPUT_PP	Active-high	Drives stop relay via transistor / driver IC
8	Alarm Relay	PC2	Output (digital)	GPIO_OUTPUT_PP	Active-high	External siren / panel alarm
9	Buzzer	PC3	Output	GPIO_OUTPUT_PP	Active-	Local audible alert

#	Signal	Pin	Direction	Mode	Active Level	Notes
			(digital)		high	
10	Fault LED	PC4	Output (digital)	GPIO_OUTPUT_PP	Active-high	Local visual fault indicator
11	UART TX (to ESP32)	PA9	Output	USART1_TX	-	Alternate function, sends telemetry to ESP32
12	UART RX (from ESP32)	PA10	Input	USART1_RX	-	Alternate function, receives commands from ESP32

4. ADC Input Plan

Three analog channels are sampled continuously and converted to engineering units before being passed to the health logic.

Parameter	Sensor / Source	Raw ADC Range	Engineering Range
Motor Current	ACS712 / shunt	0 - 4095 (12-bit)	0 - 20 A
Motor Temperature	NTC thermistor	0 - 4095 (12-bit)	20 - 120 C
Vibration Level	SW-420 / piezo	0 - 4095 (12-bit)	0 - 100 %

Linear scaling is used for all three channels:

```
engineering_value = map(raw_adc, 0, 4095, range_min, range_max)
```

In the reference ESP32 implementation this is realized with the Arduino map() function. On the STM32 side, the equivalent fixed-point or floating-point scaling is applied after each ADC conversion completes, prior to threshold comparison in the health logic.

5. Runtime Calculation Logic

Cumulative runtime is tracked from system start using a millisecond tick counter, converted to hours for telemetry reporting:

```
runtimeHours = (currentTick - startTick) / 3600000.0
```

This value increases continuously while the controller is powered. A production refinement (see Section 9) would gate this increment on the start/stop feedback signal so that runtime reflects actual motor-on time rather than controller-on time.

6. Motor Health and Fault Logic

The health classification uses two threshold tiers evaluated against all three monitored parameters. A FAULT condition takes priority over a WARNING condition, which in turn takes priority over GOOD.

Health State	Current	Temperature	Vibration
FAULT	> 18 A	> 90 C	> 80 %
WARNING	> 15 A	> 70 C	> 60 %
GOOD	<= 15 A	<= 70 C	<= 60 %

Output behavior per health state:

Health State	Motor LED	Alarm Relay	Buzzer	Fault LED
GOOD	ON	OFF	OFF	OFF
WARNING	ON	ON	OFF	OFF
FAULT	OFF	ON	ON	ON

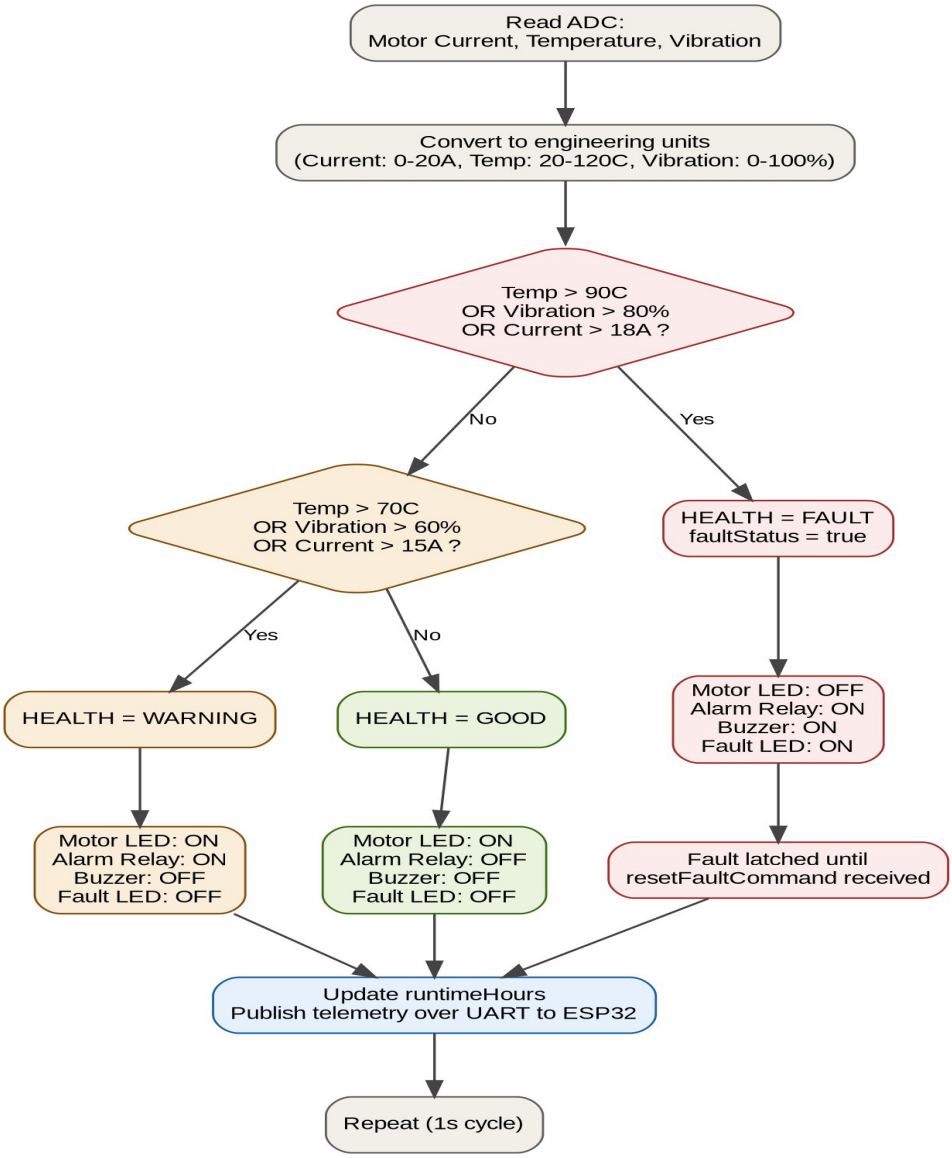

Figure 2: Motor health and fault decision flowchart

7. ESP32 Firebase Integration

The ESP32 layer is responsible for periodically uploading telemetry to Firebase and polling for remote commands. The schema below reflects the fields required by the task brief.

7.1 Firebase Data Upload Schema

Field	Type	Description
motorCurrent	Float (A)	Current ADC reading mapped 0-20A

Field	Type	Description
motorTemperature	Float (C)	Temperature ADC reading mapped 20-120C
vibration	Float (%)	Vibration ADC reading mapped 0-100%
runtimeHours	Float (hrs)	Elapsed runtime since power-on
motorStatus	String	Reflects current motor run state (start/stop feedback)
faultStatus	Boolean	true when health logic detects FAULT condition
healthStatus	String	GOOD / WARNING / FAULT
alertStatus	String	Human-readable summary alert for dashboard

7.2 Firebase Command Read Schema

Field	Description
motorStartCommand	Cloud-issued command to energize motor start relay
motorStopCommand	Cloud-issued command to energize motor stop relay
alarmCommand	Manual override to trigger alarm relay
buzzerCommand	Manual override to trigger buzzer
resetFaultCommand	Clears the latched fault state
modeCommand	Switches between AUTO and MANUAL control mode

7.3 Simulated Outputs

- Motor Run LED
- Alarm LED
- Buzzer LED
- Fault LED

8. Reference Implementation

The following code was developed and compiled for this submission. The ESP32 sketch simulates the sensor inputs using potentiometers (suitable for Wokwi simulation) and implements the health/fault logic described in Section 6. The STM32 program, compiled successfully in STM32CubeIDE, demonstrates the equivalent decision logic structure targeted at the STM32 core.

8.1 ESP32 Firmware (Arduino, sketch.ino)

Pin assignment used in this simulation build:

- Motor Current — GPIO 34 (analog input)
- Motor Temperature — GPIO 35 (analog input)
- Vibration — GPIO 32 (analog input)
- Motor LED — GPIO 18
- Alarm LED — GPIO 19
- Buzzer LED — GPIO 21
- Fault LED — GPIO 22

```
#define CURRENT_PIN     34
#define TEMP_PIN        35
#define VIBRATION_PIN   32
#define MOTOR_LED       18
#define ALARM_LED       19
#define BUZZER_LED      21
#define FAULT_LED       22
unsigned long startTime = 0;
float runtimeHours = 0;
float motorCurrent;
float motorTemp;
float vibration;
bool faultStatus = false;
String healthStatus = "GOOD";
void setup() {
 Serial.begin(115200);
 pinMode(MOTOR_LED, OUTPUT);
 pinMode(ALARM_LED, OUTPUT);
 pinMode(BUZZER_LED, OUTPUT);
 pinMode(FAULT_LED, OUTPUT);
 startTime = millis();
 Serial.println("Motor Health Monitoring Controller");
}
void loop() {
 // Read potentiometers
 int currentADC = analogRead(CURRENT_PIN);
 int tempADC = analogRead(TEMP_PIN);
 int vibrationADC = analogRead(VIBRATION_PIN);
 // Convert to engineering values
 motorCurrent = map(currentADC, 0, 4095, 0, 20);     // 0-20 A
 motorTemp    = map(tempADC, 0, 4095, 20, 120);      // 20-120 C
 vibration    = map(vibrationADC, 0, 4095, 0, 100);  // 0-100 %
 // Runtime calculation
 runtimeHours = (millis() - startTime) / 3600000.0;
 faultStatus = false;
 // Health Logic
 if (motorTemp > 90 || vibration > 80 || motorCurrent > 18)
 {
  healthStatus = "FAULT";
  faultStatus = true;
```

```
 }
 else if (motorTemp > 70 || vibration > 60 || motorCurrent > 15)
 {
  healthStatus = "WARNING";
 }
 else
 {
  healthStatus = "GOOD";
 }
 // Output Control
 if (healthStatus == "GOOD")
 {
  digitalWrite(MOTOR_LED, HIGH);
  digitalWrite(ALARM_LED, LOW);
  digitalWrite(BUZZER_LED, LOW);
  digitalWrite(FAULT_LED, LOW);
 }
 else if (healthStatus == "WARNING")
 {
  digitalWrite(MOTOR_LED, HIGH);
  digitalWrite(ALARM_LED, HIGH);
  digitalWrite(BUZZER_LED, LOW);
  digitalWrite(FAULT_LED, LOW);
 }
 else
 {
  digitalWrite(MOTOR_LED, LOW);
  digitalWrite(ALARM_LED, HIGH);
  digitalWrite(BUZZER_LED, HIGH);
  digitalWrite(FAULT_LED, HIGH);
 }
 // Serial Monitor
 Serial.println("--------------------------------");
 Serial.print("Motor Current : ");
 Serial.print(motorCurrent);
 Serial.println(" A");
 Serial.print("Temperature   : ");
 Serial.print(motorTemp);
 Serial.println(" C");
 Serial.print("Vibration     : ");
 Serial.print(vibration);
 Serial.println(" %");
 Serial.print("Runtime Hours : ");
 Serial.println(runtimeHours, 4);
 Serial.print("Health Status : ");
 Serial.println(healthStatus);
 Serial.print("Fault Status  : ");
 Serial.println(faultStatus);
 delay(1000);
}
```

8.2 STM32 Firmware (STM32CubeIDE, main.c)

This minimal C program was compiled successfully in STM32CubeIDE to validate the core decision-logic structure on the target toolchain prior to full HAL-based GPIO/ADC integration.

```
#include <stdio.h>

int main(void)
{
   int motorCurrent = 10;
   int motorTemp = 45;
   int vibration = 20;

   while(1)
   {
      if(motorTemp > 90 || vibration > 80 || motorCurrent > 18)
      {
         //  sample fault condition
      }
      else if(motorTemp > 70 || vibration > 60 || motorCurrent > 15)
      {
         // sample warning condition
      }
      else
      {
         // if its in good condition
      }
   }

   return 0;
}
```

A compilation screenshot from STM32CubeIDE confirming successful build is included as a separate submission attachment, per the task requirements.

9. Design Notes and Recommendations

The following observations are noted for transparency and reflect the difference between this simulation-stage build and a production-ready system:

- Fault latching: the current reference sketch resets faultStatus to false at the start of every loop iteration before re-evaluating thresholds, so a FAULT condition clears automatically once readings drop back below threshold. A production version should latch faultStatus = true persistently and only clear it on an explicit resetFaultCommand, as described in Section 6, to avoid masking transient but real fault events.
- Runtime gating: runtime currently accumulates from power-on rather than from actual motor-on time. Gating the increment on the debounced start/stop feedback signal (Section on digital inputs) would give a more meaningful runtime-hours metric.

- Signal isolation: the ADC inputs in this simulation use potentiometers as stand-ins for ACS712, NTC, and SW-420 sensor outputs. A physical build would require appropriate signal conditioning (voltage dividers, amplification, or isolation) before each STM32 ADC pin, as outlined in Section 4.
- STM32-ESP32 link: the planning assumes UART at a standard baud rate (commonly 115200) carrying a simple delimited telemetry/command frame; framing and checksum design were out of scope for this submission but are recommended for production reliability.

10. Submission Checklist

- STM32 planning PDF — this document
- STM32 block diagram — Figure 1
- STM32 GPIO allocation table — Section 3
- STM32 firmware flowchart — Figure 2
- STM32 compilation screenshot — attached separately
- ESP32 source code — Section 8.1
- ESP32 simulator link — attached separately
- Firebase database screenshot — attached separately
- Firebase command control screenshot — attached separately
- Serial monitor screenshot — attached separately
- Working video link — attached separately
- GitHub repository link — attached separately
- Final explanation PDF — this document